

How to Use the Part 1 Tools

This directory contains both the theoretical derivation for the Practice Final and a generalized Python tool to automate the process for any future system.

1. Automated Derivation (`symbolic_dynamics_deriver.py`)

This script uses **SymPy** to perform the calculus for the Euler-Lagrange method.

When to use this:

Use this during the actual final if you are given a new mechanical system (e.g., a cart-pendulum, a double pendulum, or a mass-spring-damper) and need to find the Equations of Motion (EOM) without making manual differentiation errors.

How to adapt it for a new system:

Open `symbolic_dynamics_deriver.py` and modify the following sections:

1. **Variables (`__init__`):**
 - Change `self.theta` to your coordinate (e.g., `self.x = sp.Function('x')(self.t)`).
 - Add/remove parameters (e.g., `self.M`, `self.L`, `self.k`).
2. **Energy Equations (`define_energies`):**
 - Update `self.K` (Kinetic Energy).
 - Update `self.P` (Potential Energy).
 - *Note: SymPy handles the derivatives of the Lagrangian automatically.*
3. **Forces (`define_generalized_forces`):**
 - Update `self.Q_theta` to include any external torques, forces, or friction terms ($b\dot{q}$).

4. **Run:**

```
python symbolic_dynamics_deriver.py
```

The script will print the Euler-Lagrange equation and the solved expression for the acceleration (e.g., $\ddot{\theta} = \dots$).

2. Theoretical Notes

- **Equations_of_Motion_Steps.md:** A checklist of the 6 steps required by the final exam to show your work. Follow this when writing your solution on the PDF/Word document.
- **Energy_Equations.md:** Reference for how the kinetic and potential energy were defined for the Rod-Mass system.

3. Implementation Checklist (Simulation)

Once you have the EOM ($\ddot{\theta}$), you must update the simulation code to see the results:

1. **L_rodmass/params.py**: Ensure all constants (m, ℓ, g, k_1, k_2, b) and simulation parameters (t_s, τ_{max}) are defined.
2. **L_rodmass/dynamics.py**:
 - In `f(x, u)`, extract `theta = x[0]` and `thetadot = x[1]`.
 - Calculate `theta_ddot` using the expression found by the symbolic script.
 - Return `np.array([thetadot, theta_ddot])`.
 - In `h(x)`, return the measured output (usually `np.array([theta])`).

Summary of Files

File	Purpose
<code>symbolic_dynamics_deriver.py</code>	Automated calculus. Generates $\ddot{\theta}$ symbolically.
<code>Equations_of_Motion_Steps.md</code>	Final Exam Guide. Steps to show work on the test.
<code>Energy_Equations.md</code>	Reference. Specific energy formulas for Part 1.
<code>Problem_Description.md</code>	Context. Recap of the Rod-Mass problem.